ChargeGuard AI — Project Status & Plan Analysis

PayPal AI Hackathon 2026 | Technical Audit, Work Accounting & Forward Roadmap

Audit Date: 7 Nov 2026
Event Deadline: 12 Nov 2026, 12:00 PT (13 Nov 01:30 IST)
Author: Antigravity Pair Programmer / Pratik Sinha

Day 1 & 2	19 Events	7 Tables	5 Days Left
PHASE COMPLETED (~90%)	VERIFIED SANDBOX WEBHOOKS	STRICT SQLITE SCHEMA	UNTIL FINAL SUBMISSION

1. Executive Summary & Core Paradigm

ChargeGuard AI is an automated dispute prevention and deflection platform designed specifically for small merchants who sell subscriptions via PayPal. Small businesses frequently lose revenue and customer goodwill from silent renewal failures, accidental double charges, and tight response windows on formal PayPal disputes.

ChargeGuard operates on two fronts: **a customer-facing portal** where buyers describe grievances in plain language (preventing escalation to PayPal), and **an owner dashboard** where an AI engine reads complaints, cross-references transaction history, checks required evidence checklists, and drafts resolutions.

> **Architectural Axiom: The AI Never Touches Money**
> The single foundational rule governing ChargeGuard is that the AI proposes, a human owner approves, and only then does the backend execute against PayPal's write endpoints. A single unified gate function guarantees that no refund, claim concession, or message is ever fired autonomously without explicit human approval.

2. Detailed Work Accounting: Days 1 & 2

The project plan budgeted Days 1–2 for the core plumbing: sandbox authentication, webhook reception and signature verification, database schema, seeder scripts, and buyer-side dispute drivers. Below is the precise forensic accounting of what was planned versus what has been built and verified on disk.

COMPONENT / PLAN ITEM	STATUS	IMPLEMENTATION DETAILS	FORENSIC VERIFICATION & EVIDENCE
PayPal Sandbox Credentials & Accounts	DONE	Configured US Business & US Personal sandbox accounts and REST App (replacing initial Indian accounts which blocked recurring USD).	Client ID & Secret active in `.env`. OAuth bearer token generation functioning with 60s pre-expiry cache in `paypal.ts`.
Database Schema (7 Tables)	DONE	SQLite database created via `better-sqlite3` in WAL mode. Contains strict SQL CHECK constraints on case states, action states, and action types.	Implemented in `server/src/db.ts`. Unit tests in `db.test.ts` pass, verifying constraint enforcement and event idempotency.
Webhook Ingestion & Signature Verification	DONE	Fastify route `POST /webhooks/paypal`. Performs live cryptographic verification via PayPal's `/v1/notifications/verify-webhook-signature` endpoint.	Verified live through public tunnel. Unsigned or corrupted payloads return clean 400. 19 signed sandbox webhook events stored in `webhook_events` table.

COMPONENT / PLAN ITEM	STATUS	IMPLEMENTATION DETAILS	FORENSIC VERIFICATION & EVIDENCE
Subscription Seeder Script	DONE	Automated CLI script creating product, monthly billing plan ($9.99), and customer subscription with a custom ID (`customId = cg-${Date.now()}`).	Implemented in `server/src/seed.ts` . Successfully executed 3 times, generating live PayPal subscriptions (e.g. `I-UHW7LKXV1YTL` , `I-BN5BHFNLUH49`).
Multi-Client Auth & Auth-Assertion	DONE	Added per-app token cache and base64url-encoded `PayPal-Auth-Assertion` header generator to act on behalf of buyer accounts.	Implemented in `server/src/paypal.ts` (`Creds` type, `tokens` map, `authAssertion()` helper).
Sandbox Dispute Driver Script	CODED / CONFIG	CLI tool to create buyer disputes (`npm run dispute create`) and adjudicate outcomes in merchant favor (`npm run dispute settle`).	Implemented in `server/src/dispute.ts` and `package.json` . *Pending:* buyer sandbox credentials in `.env` to fire first live dispute.
AI Integration Helper & Schema Validator	CODED / TESTING	OpenAI-compatible wrapper `generateJson()` using Zod schema to enforce strict JSON structure and eliminate unformatted prose.	Implemented in `server/src/ai.ts` ; test driver in `server/src/ai-check.ts` . Live API key populated; model endpoint compatibility identified.
Repository Cleanup & Git Hygiene	DONE	Removed accidental CI workflow commits (`deno.yml` , `webpack.yml`); secured private plan specifications; maintained clean git status.	Clean working branch on GitHub (`f982ef6`). Sensitive credentials prevented from staging via `.gitignore` .

3. In-Depth Analysis of Current Gaps & Blockers

While the underlying infrastructure is robust, closing out Day 2 and entering Day 3 requires resolving three specific configuration bottlenecks:

> **1. Missing Buyer Sandbox Credentials (Dispute Driver)**
> In `.env`, the variables `BUYER_CLIENT_ID`, `BUYER_CLIENT_SECRET`, and `BUYER_EMAIL` are currently empty. **Impact:** Running `npm run dispute create` will throw an authentication error. To complete the automated dispute demo cycle, the developer needs to generate sandbox API credentials inside the PayPal Developer Portal for the buyer personal account and populate these three keys.

> **2. AI Model Endpoint Configuration & Stability**
> The AI helper targets Google's OpenAI-compatible endpoint (`https://generativelanguage.googleapis.com/v1beta/openai`). The test check was configured with `AI_MODEL=gemini-3.8-flash`, which resulted in intermittent 503 high-demand responses. **Remediation:** Switching to GA model strings such as `gemini-2.5-flash` or incorporating a retry/backoff mechanism in `server/src/ai.ts` will ensure uninterrupted classification and drafting during evaluation.

> **3. Transaction Search Permission Latency**
> The PayPal Transaction Search permission takes up to 9 hours to activate on a newly provisioned REST App. Until approved, list-transaction calls fail. **Architecture Resilience:** The architecture correctly designates Webhooks as the authoritative real-time source of truth; Transaction Search is retained solely for historical backfill.

4. What is in the Plan Next (Days 3 to 10)

The remaining 8 days are structured into clearly defined functional blocks to guarantee a bulletproof working demo ahead of the November 12 submission deadline.

> **Days 3–4: Case Engine, Rules Layer & Dispute Ingestion (Immediate Next Phase)**
>
> - **Webhook Dispatcher:** Build an asynchronous event processor that unpacks stored events from `webhook_events` into `subscriptions` and `transactions` tables. Handles out-of-order events gracefully (e.g. `PAYMENT.SALE.COMPLETED` landing before `BILLING.SUBSCRIPTION.ACTIVATED`).
> - **Deterministic SQL Rules Engine:** Execute rule checks before invoking AI:
>   1. *Duplicate Charge Detector:* SQL self-join over `transactions` matching same payer, exact same amount, within a 48-hour window.
>   2. *Failed Payment Monitor:* Tracks retry counts and dunning stage directly from subscription status changes.
>   3. *Deadline Clock:* Computes hours remaining until `seller_response_due_date`; dynamically orders merchant queue.
> - **Dispute Ingestion:** On `CUSTOMER.DISPUTE.CREATED`, immediately issue `GET /v1/customer/disputes/{id}` to retrieve authoritative seller response due date and stage.

> **Days 5–6: The Three Bounded AI Jobs (Schema-Constrained)**
>
> All AI operations return strictly validated Zod schemas. Free-form text never enters the database directly.
>
> - **Job 1 — Intake Triage:** Takes complaint prose and transaction context; outputs `{ reason_code, transaction_ref, remedy_sought, urgency, summary }`. Reason codes: `INR`, `SNAD`, `UNAUTHORIZED`, `DUPLICATE`, `SUBSCRIPTION_CANCEL`.
> - **Job 2 — Evidence Assessment:** Checks merchant evidence against PayPal's explicit dispute guidelines.
>   *Crucial Design Decision:* **No fabricated win percentages.** Judges with payment domain knowledge reject synthetic probabilities. ChargeGuard outputs factual completeness: `{ present: [...], missing: [...], strength: "4 of 6", rationale }`.
> - **Job 3 — Response Drafting:** Generates two prompt variants: customer resolution messaging (for inquiries and portal deflection) and formal evidence representation (for escalated PayPal claims).

- **The Approval Gate:** Implementation of `POST /api/actions/:id/approve` and `reject`. Enforces that no PayPal refund or evidence submission is called unless `actions.status = 'approved'`.

Days 7–8: Frontends (Owner Dashboard & Customer Portal)

- **Owner Dashboard:** Priority queue sorted by urgency/deadline. Case detail view presenting customer statement, AI triage summary, evidence checklist with missing item badges, editable draft message, and explicit "Approve & Execute" action button.
- **Customer Portal (3 Screens):** Accessible via signed, expiring token link (no customer passwords required). Shows current subscription plan, next billing date, payment history, plain-English complaint intake box, and merchant communication thread.

Day 9: Financial Recovery Ledger & 3-Minute Video Production

- **Recovery Ledger:** Live ROI metrics widget displaying total cases deflected before dispute opening, disputes won, and dollar value protected.
- **3-Minute Video Recording:** Scripted to the second: (0:00–0:20 Problem intro) → (0:20–0:40 Customer portal filing) → (0:40–1:10 Dashboard approval & pre-dispute resolution) → (1:10–1:30 "No dispute was ever opened" deflection impact) → (1:30–2:10 Live sandbox dispute settlement) → (2:10–2:45 Ledger & architecture summary).

Day 10: Hosting, Buffer, Open Source Licensing & Final Submission

- Deploy backend and frontend to stable hosting (Render with sponsor credits) to replace temporary cloudflared tunnel.
- Final documentation review, MIT license verification, Devpost submission text, and URL checks ahead of the deadline.

5. Strategic Assessment & Hackathon Winning Criteria Alignment

The PayPal AI Hackathon evaluates projects across five equally weighted criteria. Here is how ChargeGuard's technical decisions specifically score against them:

EVALUATION CRITERIA	PROJECT ALIGNMENT & TECHNICAL EXECUTION	COMPETITIVE ADVANTAGE
Technological Implementation	Integrates 4 distinct PayPal API surfaces (Subscriptions, Transactions, Disputes, and Webhooks) with live signature verification and an event-driven state machine.	Surpasses typical hackathon submissions that merely add a ChatGPT wrapper to a checkout button.
Innovation & Framing	**Deflection Framing:** Resolves problems before PayPal ever registers a dispute. Prevents dispute processing fees ($15–$25 per instance) and maintains merchant account health.	Existing tools are reactive and merchant-only. A two-sided system intercepting issues early is novel.
Potential Impact	Tailored for solo founders and micro-merchants lacking full-time chargeback support staff. Transparent evidence scoring saves hours of manual paperwork.	Direct, measurable financial ROI through the Recovery Ledger.
Design & User Experience	Frictionless, passwordless customer portal; triage view with editable drafts ensuring humans stay in control without feeling overwhelmed.	Clean separation of deterministic rules (speed) vs LLM reasoning (drafting).
Presentation & Clarity	Crisp 3-minute video showing end-to-end deflection and dispute resolution with real sandbox accounts and webhook logs.	Defensible technical explanations (e.g. why we avoid arbitrary win percentages).

6. Actionable Next Steps (Immediate Checklist)

1. **Fill Buyer Sandbox Credentials in `.env`:**
   Obtain `BUYER_CLIENT_ID` and `BUYER_CLIENT_SECRET` for the sandbox Personal account from the PayPal Developer Portal.
2. **Execute Dispute Test Driver:**
   Run `npm run dispute create <sale_id>` to trigger a sandbox dispute and verify that a `CUSTOMER.DISPUTE.CREATED` webhook lands in `webhook_events`.
3. **Initiate Day 3 Case Engine Implementation:**
   Create the background event processor to consume webhook events, populate `cases`, and run deterministic SQL rule checks.
4. **Lock AI Model String:**
   Ensure `AI_MODEL=gemini-2.5-flash` (or verified stable equivalent) in `.env` and confirm `npm run ai:check` outputs `{ reason_code: "DUPLICATE" }` consistently.